Ubuntu 22.04 单Master节点部署Kubernetes 1.28 + 可视化Dashboard

一、系统环境准备（所有节点执行）

1. 关闭Swap分区（必须）

Kubernetes要求永久关闭Swap，否则kubelet无法正常启动。

```
# 临时关闭Swap
sudo swapoff -a

# 永久关闭Swap（注释掉fstab中的swap行）
sudo sed -i '/swap/s/^/#/' /etc/fstab

# 验证Swap已关闭（输出应为空）
cat /etc/fstab | grep swap
free -h
```

2. 关闭防火墙

```
sudo ufw disable
sudo systemctl stop ufw
sudo systemctl disable ufw
```

3. 加载内核模块并配置系统参数

```
# 加载必要的内核模块
sudo modprobe overlay
sudo modprobe br_netfilter

# 配置sysctl参数
cat <<EOF | sudo tee /etc/sysctl.d/k8s.conf
net.bridge.bridge-nf-call-iptables  = 1
net.bridge.bridge-nf-call-ip6tables = 1
net.ipv4.ip_forward                 = 1
EOF
```

```
# 应用参数
sudo sysctl --system
```

4. 设置主机名和hosts解析

```
# 设置主机名（替换为你的主机名）
sudo hostnamectl set-hostname k8s-master

# 配置hosts文件（添加本机IP和主机名映射，请将192.168.1.100替换为你服务器的实际内网IP地址）
echo "192.168.1.100 k8s-master" | sudo tee -a /etc/hosts
```

二、安装Containerd容器运行时

Kubernetes 1.24+不再默认使用Docker作为容器运行时，推荐使用Containerd。

1. 安装依赖

```
sudo apt update && sudo apt install -y ca-certificates curl gnupg lsb-release
```

2. 添加Docker官方GPG密钥和apt源

```
sudo mkdir -p /etc/apt/keyrings
curl -fsSL https://download.docker.com/linux/ubuntu/gpg | sudo gpg --dearmor -o /etc/apt/keyrings/docker.gpg

echo "deb [arch=$(dpkg --print-architecture) signed-by=/etc/apt/keyrings/docker.gpg] https://download.docker.com/linux/ubuntu $(lsb_release -cs) stable" | sudo tee /etc/apt/sources.list.d/docker.list > /dev/null
```

3. 安装Containerd

```
sudo apt update
```

```
sudo apt install -y containerd.io
```

4. 配置Containerd

```
# 生成默认配置文件
sudo containerd config default | sudo tee /etc/containerd/config.toml

# 修改配置，使用systemd作为cgroup驱动（必须）
sudo sed -i 's/SystemdCgroup = false/SystemdCgroup = true/' /etc/containerd/config.toml

# 重启Containerd并设置开机自启
sudo systemctl restart containerd
sudo systemctl enable containerd

# 验证Containerd状态
sudo systemctl status containerd
```

三、安装Kubernetes 1.28组件

1. 添加Kubernetes官方apt源

```
# 添加Google Cloud公钥
curl -fsSL https://pkgs.k8s.io/core:/stable:/v1.28/deb/Release.key | sudo gpg --dearmor -o /etc/apt/keyrings/kubernetes-apt-keyring.gpg

# 添加Kubernetes 1.28版本源
echo "deb [signed-by=/etc/apt/keyrings/kubernetes-apt-keyring.gpg] https://pkgs.k8s.io/core:/stable:/v1.28/deb/ /" | sudo tee /etc/apt/sources.list.d/kubernetes.list
```

2. 安装指定版本的kubeadm、kubelet、kubectl

```
sudo apt update
sudo apt install -y kubelet=1.28.10-1.1 kubeadm=1.28.10-1.1 kubectl=1.28.10-1.1

```

```
# 锁定版本，防止自动更新
sudo apt-mark hold kubelet kubeadm kubectl

# 验证安装
kubeadm version
kubectl version --client
```

四、初始化Kubernetes Master节点

1. 拉取所需镜像（可选，提前拉取加快初始化速度）

```
sudo kubeadm config images pull --kubernetes-version=1.28.10
```

2. 执行初始化命令

```
sudo kubeadm init \
  --kubernetes-version=1.28.10 \
  --apiserver-advertise-address=192.168.1.100 \
  --pod-network-cidr=10.244.0.0/16 \
  --service-cidr=10.96.0.0/12 \
  --cri-socket=unix:///var/run/containerd/containerd.sock
```

参数说明：

- `--apiserver-advertise-address` ： 替换为你的服务器内网IP
- `--pod-network-cidr` ： Pod网络段，后续安装Calico网络插件需要使用这个网段
- `--service-cidr` ： Service网络段

3. 配置kubectl访问权限

```
mkdir -p $HOME/.kube
sudo cp -i /etc/kubernetes/admin.conf $HOME/.kube/config
sudo chown $(id -u):$(id -g) $HOME/.kube/config

# 验证kubectl连接
```

```
kubectl get nodes
```

此时你会看到Master节点状态为 `NotReady` ，这是因为还没有安装网络插件。

五、安装Calico网络插件

Calico是Kubernetes最常用的网络插件之一，性能稳定且功能丰富。

```
# 下载Calico 3.26版本（兼容K8s 1.28）
kubectl apply -f https://raw.githubusercontent.com/projectcalico/calico/v3.26.4/manifests/calico.yaml

# 等待Calico Pod全部就绪（大约需要1-2分钟）
kubectl wait --for=condition=ready pod -l k8s-app=calico-node -n kube-system --timeout=300s
kubectl wait --for=condition=ready pod -l k8s-app=calico-kube-controllers -n kube-system --timeout=300s

# 再次验证节点状态（此时应该变为Ready）
kubectl get nodes
```

六、移除Master节点污点（单节点部署必须）

默认情况下，Kubernetes不会在Master节点上调度Pod。单节点部署需要移除这个污点：

```
kubectl taint nodes k8s-master node-role.kubernetes.io/control-plane-
```

输出 `node/k8s-master untainted` 表示成功。

七、部署Kubernetes Dashboard可视化界面

1. 部署Dashboard v2.7.0（兼容K8s 1.28）

```
#这步网络问题会失败
kubectl apply -f https://raw.githubusercontent.com/kubernetes/dashboard/v2.7.0/aio/deploy/recommended.yaml

#创建本地yaml文件
```

```
vim dashboard-aliyun.yaml
```

2. 粘贴以下完整内容到dashboard-aliyun.yam（已替换所有镜像为阿里云）

```
# Copyright 2017 The Kubernetes Authors.
# Licensed under the Apache License, Version 2.0
# 已将所有镜像替换为阿里云国内镜像：registry.cn-hangzhou.aliyuncs.com/google_containers

apiVersion: v1
kind: Namespace
metadata:
  name: kubernetes-dashboard

---
apiVersion: v1
kind: ServiceAccount
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard

---
kind: Service
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
spec:
  ports:
    - port: 443
      targetPort: 8443
      nodePort: 30000  # 已提前设置NodePort端口，无需后续修改
  type: NodePort  # 已改为NodePort，直接通过IP:30000访问
  selector:
    k8s-app: kubernetes-dashboard

```

```yaml
---
apiVersion: v1
kind: Secret
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-certs
  namespace: kubernetes-dashboard
type: Opaque

---
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-csrf
  namespace: kubernetes-dashboard
data:
  csrf: ""

---
apiVersion: v1
kind: Secret
type: Opaque
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-key-holder
  namespace: kubernetes-dashboard

---
kind: ConfigMap
apiVersion: v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard-settings
  namespace: kubernetes-dashboard

---
```

```
kind: Role
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
rules:
  - apiGroups: [""]
    resources: ["secrets"]
    resourceNames: ["kubernetes-dashboard-key-holder", "kubernetes-dashboard-certs", "kubernetes-dashboard-csrf"]
    verbs: ["get", "update", "delete"]
  - apiGroups: [""]
    resources: ["configmaps"]
    resourceNames: ["kubernetes-dashboard-settings"]
    verbs: ["get", "update"]
  - apiGroups: [""]
    resources: ["services"]
    resourceNames: ["heapster", "dashboard-metrics-scraper"]
    verbs: ["proxy"]
  - apiGroups: [""]
    resources: ["services/proxy"]
    resourceNames: ["heapster", "http:heapster:", "https:heapster:", "dashboard-metrics-scraper", "http:dashboard-metrics-scraper"]
    verbs: ["get"]

---
kind: ClusterRole
apiVersion: rbac.authorization.k8s.io/v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
rules:
  - apiGroups: ["metrics.k8s.io"]
    resources: ["pods", "nodes"]
    verbs: ["get", "list", "watch"]

---
apiVersion: rbac.authorization.k8s.io/v1
kind: RoleBinding
metadata:
```

```
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: Role
  name: kubernetes-dashboard
subjects:
  - kind: ServiceAccount
    name: kubernetes-dashboard
    namespace: kubernetes-dashboard

---
apiVersion: rbac.authorization.k8s.io/v1
kind: ClusterRoleBinding
metadata:
  name: kubernetes-dashboard
roleRef:
  apiGroup: rbac.authorization.k8s.io
  kind: ClusterRole
  name: kubernetes-dashboard
subjects:
  - kind: ServiceAccount
    name: kubernetes-dashboard
    namespace: kubernetes-dashboard

---
kind: Deployment
apiVersion: apps/v1
metadata:
  labels:
    k8s-app: kubernetes-dashboard
  name: kubernetes-dashboard
  namespace: kubernetes-dashboard
spec:
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      k8s-app: kubernetes-dashboard
  template:
```

```yaml
  metadata:
    labels:
      k8s-app: kubernetes-dashboard
  spec:
    securityContext:
      seccompProfile:
        type: RuntimeDefault
    containers:
      - name: kubernetes-dashboard
        # 阿里云镜像，国内极速拉取
        image: registry.cn-hangzhou.aliyuncs.com/google_containers/dashboard:v2.7.0
        imagePullPolicy: IfNotPresent
        ports:
          - containerPort: 8443
            protocol: TCP
        args:
          - --auto-generate-certificates
          - --namespace=kubernetes-dashboard
        volumeMounts:
          - name: kubernetes-dashboard-certs
            mountPath: /certs
          - mountPath: /tmp
            name: tmp-volume
        livenessProbe:
          httpGet:
            scheme: HTTPS
            path: /
            port: 8443
          initialDelaySeconds: 30
          timeoutSeconds: 30
        securityContext:
          allowPrivilegeEscalation: false
          readOnlyRootFilesystem: true
          runAsNonRoot: true
          runAsUser: 1001
    volumes:
      - name: kubernetes-dashboard-certs
        secret:
          secretName: kubernetes-dashboard-certs
      - name: tmp-volume
        emptyDir: {}
    serviceAccountName: kubernetes-dashboard
```

```
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - key: node-role.kubernetes.io/master
          effect: NoSchedule
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule

---
kind: Service
apiVersion: v1
metadata:
  labels:
    k8s-app: dashboard-metrics-scraper
  name: dashboard-metrics-scraper
  namespace: kubernetes-dashboard
spec:
  ports:
    - port: 8000
      targetPort: 8000
  selector:
    k8s-app: dashboard-metrics-scraper

---
kind: Deployment
apiVersion: apps/v1
metadata:
  labels:
    k8s-app: dashboard-metrics-scraper
  name: dashboard-metrics-scraper
  namespace: kubernetes-dashboard
spec:
  replicas: 1
  revisionHistoryLimit: 10
  selector:
    matchLabels:
      k8s-app: dashboard-metrics-scraper
  template:
    metadata:
      labels:
        k8s-app: dashboard-metrics-scraper
    spec:
```

```
      securityContext:
        seccompProfile:
          type: RuntimeDefault
      containers:
        - name: dashboard-metrics-scraper
          # 阿里云镜像，国内极速拉取
          image: registry.cn-hangzhou.aliyuncs.com/google_containers/metrics-scraper:v1.0.8
          imagePullPolicy: IfNotPresent
          ports:
            - containerPort: 8000
              protocol: TCP
          livenessProbe:
            httpGet:
              scheme: HTTP
              path: /
              port: 8000
            initialDelaySeconds: 30
            timeoutSeconds: 30
          volumeMounts:
          - mountPath: /tmp
            name: tmp-volume
          securityContext:
            allowPrivilegeEscalation: false
            readOnlyRootFilesystem: true
            runAsNonRoot: true
            runAsUser: 1001
      serviceAccountName: kubernetes-dashboard
      nodeSelector:
        kubernetes.io/os: linux
      tolerations:
        - key: node-role.kubernetes.io/master
          effect: NoSchedule
        - key: node-role.kubernetes.io/control-plane
          effect: NoSchedule
      volumes:
        - name: tmp-volume
          emptyDir: {}
```

3. 创建管理员用户并获取令牌

创建一个拥有集群管理员权限的用户，用于登录Dashboard：

```
# 创建管理员服务账户
kubectl create serviceaccount admin-user -n kubernetes-dashboard

# 绑定集群管理员权限
kubectl create clusterrolebinding admin-user-binding \
  --clusterrole=cluster-admin \
  --serviceaccount=kubernetes-dashboard:admin-user

# 获取登录令牌（有效期1小时）
kubectl -n kubernetes-dashboard create token admin-user
```

4. 访问Dashboard

```
https://你的服务器IP:30000
```

选择"令牌"登录，粘贴刚才获取的令牌即可。

八、验证部署成功

1. 检查集群组件状态

```
kubectl get cs
kubectl get pods -n kube-system
```

```
root@k8s-master:~# kubectl get cs
Warning: v1 ComponentStatus is deprecated in v1.19+
NAME                 STATUS    MESSAGE   ERROR
controller-manager   Healthy   ok
scheduler            Healthy   ok
etcd-0               Healthy   ok
```

```
root@k8s-master:~# kubectl get pods -n kube-system
NAME                                       READY   STATUS    RESTARTS   AGE
calico-kube-controllers-7c968b5878-8fr45   1/1     Running   0          21h
calico-node-jtnts                          1/1     Running   0          21h
coredns-5dd5756b68-g42ch                   1/1     Running   0          21h
coredns-5dd5756b68-tkr2n                   1/1     Running   0          21h
etcd-k8s-master                            1/1     Running   0          21h
kube-apiserver-k8s-master                  1/1     Running   0          21h
kube-controller-manager-k8s-master         1/1     Running   0          21h
kube-proxy-plgkv                           1/1     Running   0          21h
kube-scheduler-k8s-master                  1/1     Running   0          21h
```

所有Pod都应该处于 `Running` 状态。

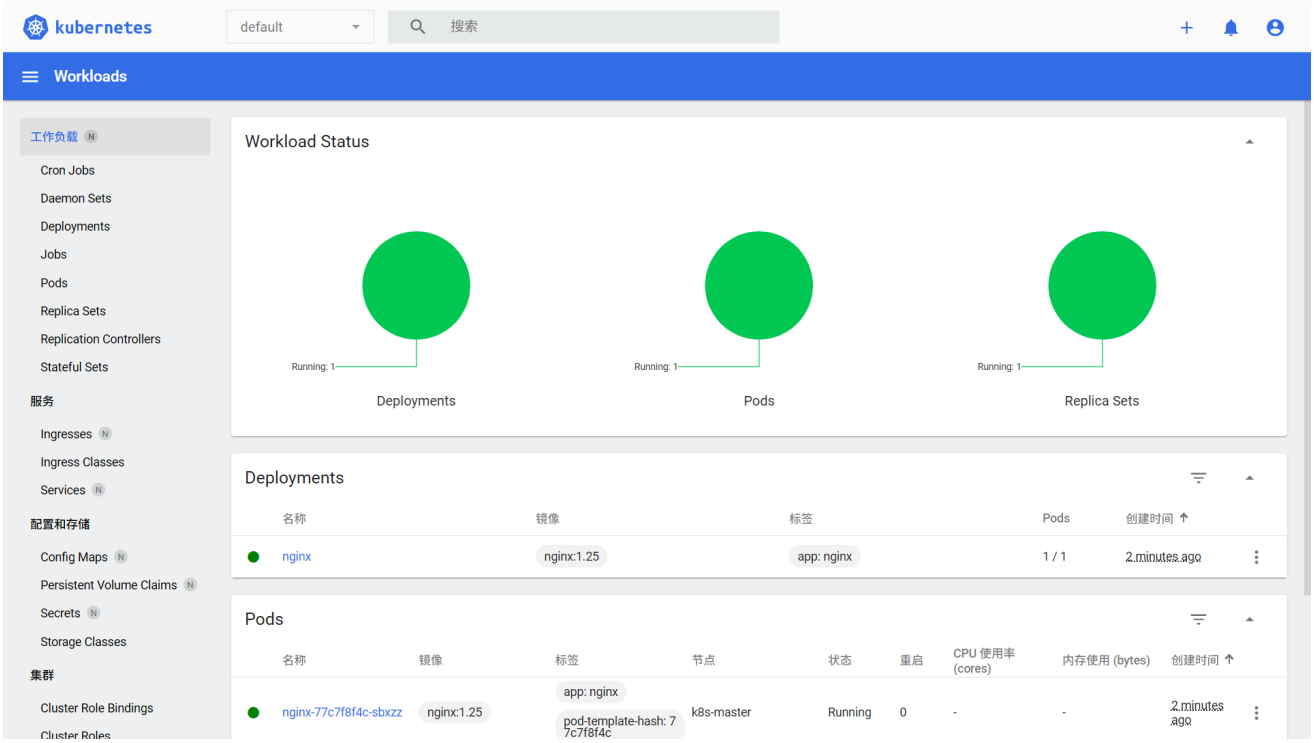
2. 测试部署一个示例应用

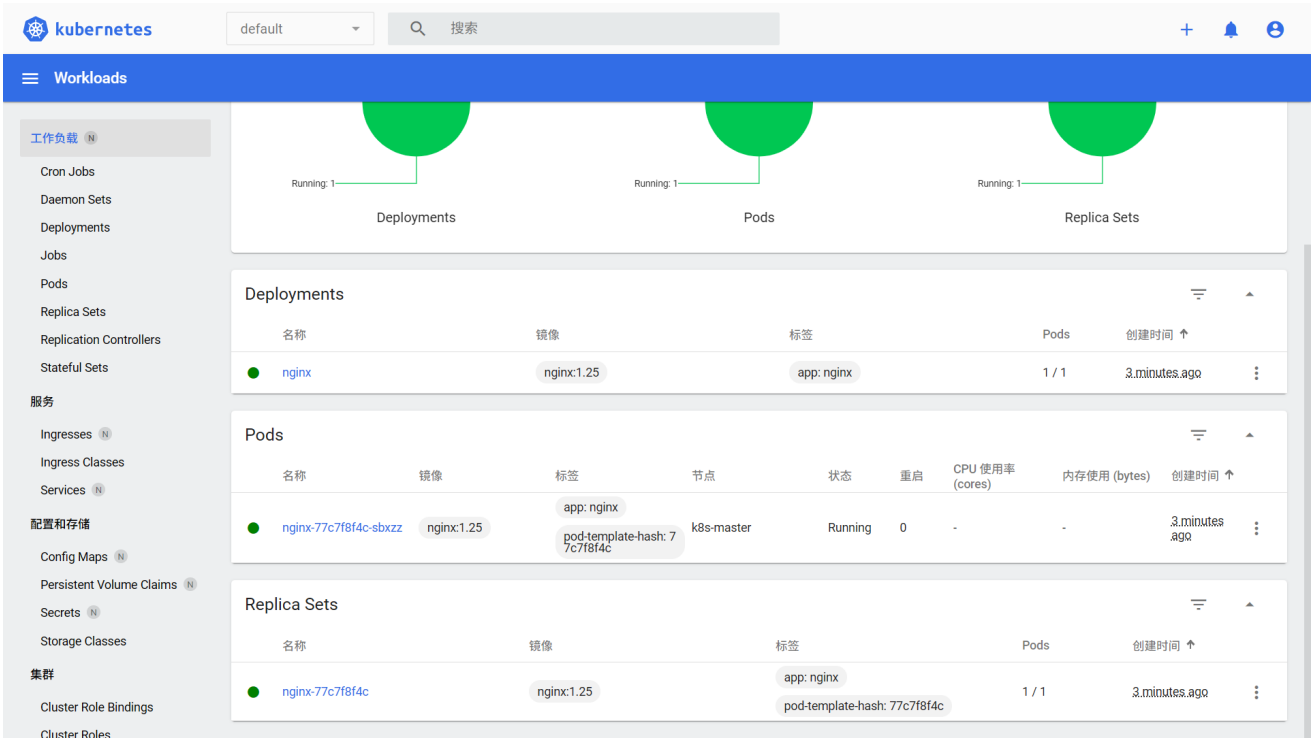
```
# 部署Nginx示例
kubectl create deployment nginx --image=nginx:1.25
kubectl expose deployment nginx --port=80 --type=NodePort

# 查看NodePort端口
kubectl get svc nginx
```

```
root@k8s-master:~# kubectl create deployment nginx --image=nginx:1.25
deployment.apps/nginx created
root@k8s-master:~# kubectl expose deployment nginx --port=80 --type=NodePort
service/nginx exposed
root@k8s-master:~# kubectl get svc nginx
NAME    TYPE       CLUSTER-IP      EXTERNAL-IP   PORT(S)        AGE
nginx   NodePort   10.99.146.253   <none>        80:32239/TCP   24s
```

然后通过 `http://你的服务器IP:NodePort端口` 访问Nginx页面，验证集群工作正常。





九、常见问题解决

1. kubeadm init失败：检查Swap是否已永久关闭，Containerd配置是否正确，系统参数是否生效。
2. 节点一直NotReady：检查Calico Pod是否正常运行，查看日志 `kubectl logs -f -n kube-system calico-node-xxx` 。
3. Dashboard无法访问：确保kubectl proxy正在运行，防火墙已关闭，URL输入正确。
4. 令牌过期：默认令牌有效期为1小时，过期后重新执行 `kubectl -n kubernetes-dashboard create token admin-user` 获取新令牌。
5. Kubernetes Dashboard默认只在集群内部可访问可以通过Dashboard的NodePort暴露方式（无需kubectl proxy，直接通过端口访问）